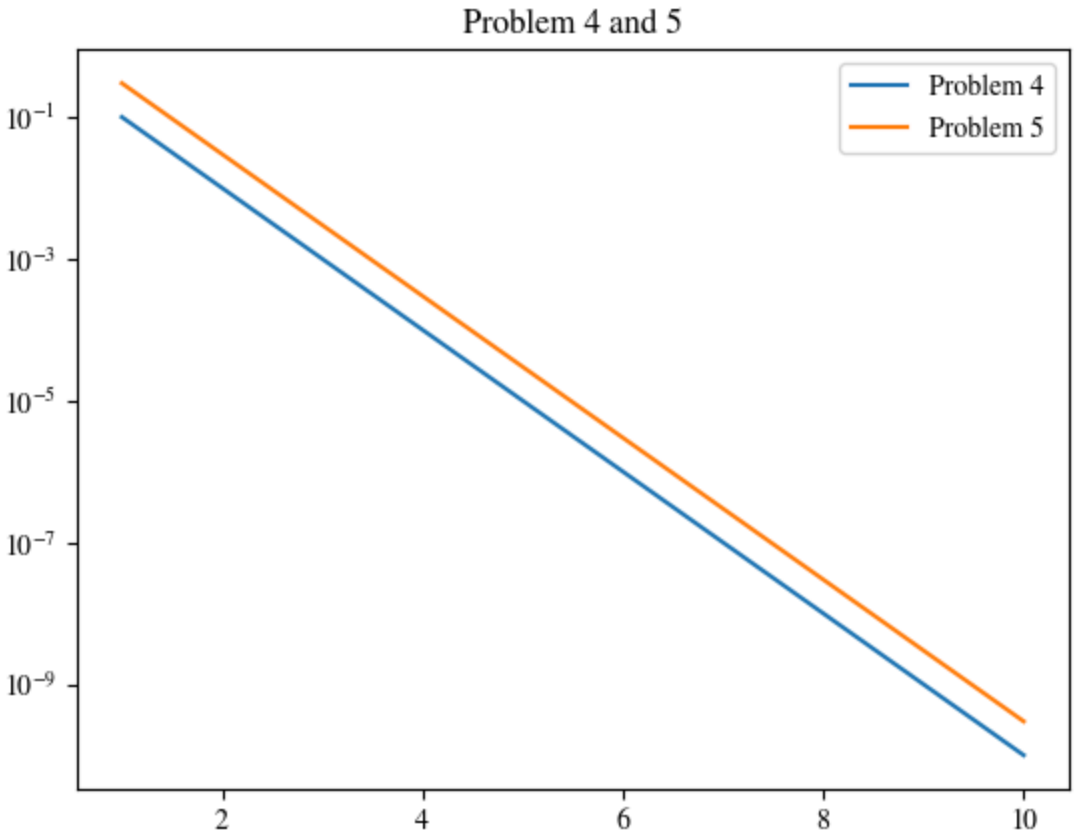
In [5]:
```
import numpy as np
import matplotlib.pylab as plt
import matplotlib
import math
%matplotlib inline
matplotlib.rcParams['mathtext.fontset'] = 'cm'
matplotlib.rcParams['font.family'] = 'STIXGeneral'
```

In [64]:
```
# Exercise 3.2

# 1
x = np.linspace(0, 4, 5)
y = np.arange(0, 5, 1)

# 2
first_3 = x[0:3]

# 3
print("The first three entries of x are", first_3)

# 4
w = 10**(-np.linspace(1,10,10))
print(w)
# The entries of w are increments of 1 x 10^-n with n moving in increments of 1

x = np.arange(1, 11, 1)
print(x)

plt.semilogy(x, w, label="Problem 4")

# 5
s = 3*w
plt.semilogy(x, s, label="Problem 5")
plt.title("Problem 4 and 5")
plt.legend()

n = 3
x = np.linspace(0,1,n)
print(x)
```

```
The first three entries of x are [0. 1. 2.]
[1.e-01 1.e-02 1.e-03 1.e-04 1.e-05 1.e-06 1.e-07 1.e-08 1.e-09 1.e-10]
[ 1  2  3  4  5  6  7  8  9 10]
[0.  0.5 1. ]
```



In [73]:
```
# Exercise 4

"""
 This program is a warm up for coding. You get used to the coding
format and practice some coding skills.
"""

###############################################
"""
Copyright (C) 2025  Adrianna M. Gillman

    This program is free software: you can redistribute it and/or modify
    it under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    This program is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
    GNU General Public License for more details.

    You should have received a copy of the GNU General Public License
    along with this program.  If not, see <https://www.gnu.org/licenses/>.
"""
###############################################


import numpy as np
import numpy.linalg as la
```

```
import math

def driver():

     n = 3
     x = np.linspace(0, 1, n)

# this is a function handle.  You can use it to define
# functions instead of using a subroutine like you
# have to in a true low level language.
     f = lambda x: x**2 + 4*x + 2*np.exp(x)
     g = lambda x: 6*x**3 + 2*np.sin(x)

#    y = f(x)
#     w = g(x)
     y = np.linspace(1, 0, n)
     w = np.linspace(0, 0, n)

     u = np.ones((2, 3))
     v = np.ones((2, 2))*5

# evaluate the dot product of y and w
     dp = dotProduct(y,w,n)

# print the output
     print('the dot product is : ', dp)
     matrix_mult(u, v)

     return

def dotProduct(x,y,n):
#   Computes the dot product of the n x 1 vectors x and y
     dp = 0.
     for j in range(n):
        dp = dp + x[j]*y[j]

     return dp


def matrix_mult(x, y):
    rows = x.shape[1]
    cols = y.shape[0]
    matrix = np.zeros((rows, cols))
    for i in range(rows):
        for j in range(cols):
            dot = dotProduct(x[i, :], y[:, j], rows)
            matrix[i, j] = dot
    return matrix


driver()
```

```
the dot product is :  0.0
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[73], line 80
     76             matrix[i, j] = dot
     77     return matrix
---> 80 driver()

Cell In[73], line 56, in driver()
     54 # print the output
     55      print('the dot product is : ', dp)
---> 56      matrix_mult(u, v)
     58      return

Cell In[73], line 75, in matrix_mult(x, y)
     73 for i in range(rows):
     74     for j in range(cols):
---> 75         dot = dotProduct(x[i, :], y[:, j], rows)
     76         matrix[i, j] = dot
     77 return matrix

Cell In[73], line 64, in dotProduct(x, y, n)
     62 dp = 0.
     63 for j in range(n):
---> 64    dp = dp + x[j]*y[j]
     66 return dp

IndexError: index 2 is out of bounds for axis 0 with size 2
```